Universidad Politécnica de Atlautla

Ingeniería en Tecnologías de la Información e Innovación Digital

Asignatura: Fundamentos de Inteligencia Artificial

Docente: Mtro. Javier García Lara

Cuatrimestre 7 · Periodo: 12 al 26 de octubre de 2026

ACTIVIDAD PARCIAL 2

Sección 3

2.3 Búsqueda local

Hill Climbing y Simulated Annealing aplicados al problema del viajante

Alumno: Cesar Natanael Rosales Vallejo · Matrícula: 24305125

Grupo: 171

Fecha de entrega: 22 de octubre de 2026

1. Planteamiento (enunciado)

Dado un conjunto de 20 ciudades con coordenadas en un plano de 100 × 100, encontrar el circuito cerrado que visite cada ciudad una sola vez y tenga la menor longitud total (problema del viajante, TSP). Se comparan dos algoritmos de búsqueda local: Hill Climbing (ascenso de colina) y Simulated Annealing (recocido simulado).

2. Datos del problema

- 20 ciudades con coordenadas aleatorias uniformes generadas con semilla 42 (reproducible).
- Distancia euclidiana entre ciudades.
- 10 ejecuciones por algoritmo con semillas distintas (100 a 109), cada una con 20 000 iteraciones.

3. Formulación

Elemento	Definición
Estado	Permutación de las 20 ciudades (orden de visita).
Función objetivo	f(ruta) = suma de distancias entre ciudades consecutivas, incluyendo el regreso al inicio. Se minimiza.
Vecindad	2-opt: se elige un segmento aleatorio de la ruta y se invierte.
Hill Climbing	Se acepta el vecino solo si mejora f; se detiene en óptimos locales (aquí, por número fijo de iteraciones).
Simulated Annealing	Se acepta un vecino peor con probabilidad exp(−Δ/T); T se reduce en cada iteración: T ← η·T.
Parámetros SA	T inicial = 100; T mínima = 0.001; η (factor de enfriamiento) = 0.9995; iteraciones = 20000.

4. Procedimiento

1. Generar las ciudades y definir la función de costo y la vecindad 2-opt.
2. Ejecutar Hill Climbing y Simulated Annealing 10 veces cada uno desde rutas iniciales aleatorias.
3. Registrar el mejor costo por iteración para graficar la convergencia.
4. Comparar costo final (mejor, media, desviación), estabilidad y tiempo.

5. Resultados

Métrica (10 ejecuciones)	Mejor	Media	Desv. estándar	Peor
Hill Climbing	352.32	356.24	4.11	362.71
Simulated Annealing	352.32	352.32	0	352.32

Tiempo promedio por ejecución: Hill Climbing 0.12 s; Simulated Annealing 0.13 s (valores dependientes del equipo).

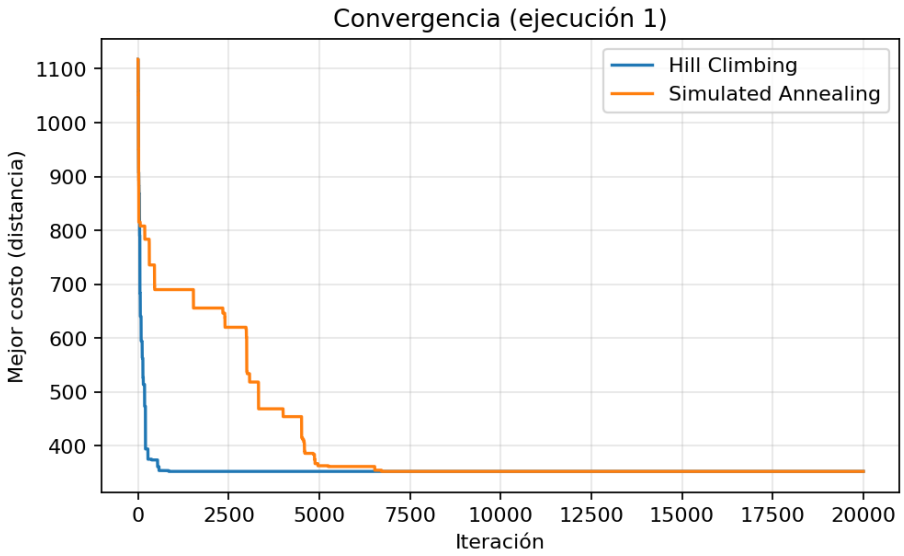


Figura 1. Convergencia del mejor costo por iteración (ejecución 1).

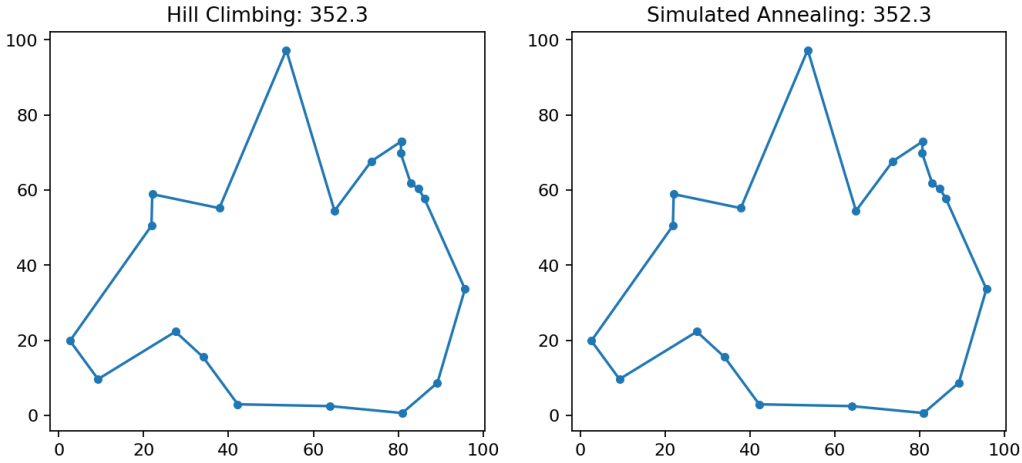


Figura 2. Rutas finales obtenidas en la ejecución 1.

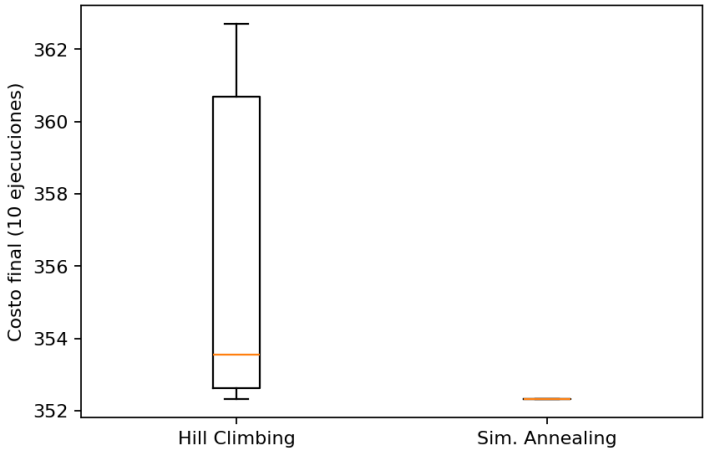


Figura 3. Distribución del costo final en 10 ejecuciones.

6. Conclusiones

Simulated Annealing llegó al mismo costo (352.32) en las 10 ejecuciones, mientras que Hill Climbing tuvo media de 356.24 y un rango de 352.32 a 362.71: en algunas ejecuciones quedó atrapado en óptimos locales. Con un tiempo de cómputo prácticamente igual, SA fue más robusto porque al principio acepta empeoramientos que le permiten salir de esos óptimos.

Hill Climbing sigue siendo útil por su simplicidad y rapidez, y con reinicios aleatorios puede igualar al mejor resultado. Como límite del estudio, se probó una sola instancia de 20 ciudades; el valor 352.32 es el mejor encontrado y no se demostró que sea el óptimo global. Los resultados pueden variar con otras instancias o parámetros de enfriamiento.

Declaración de uso de inteligencia artificial

Campo	Declaración
Herramienta	Claude (Anthropic), asistente de IA conversacional.
Prompt	Ayúdame a implementar Hill Climbing y Simulated Annealing para el problema del viajante y a comparar su convergencia.
Uso	Apoyo para estructurar la comparación, escribir las funciones de vecindad y de enfriamiento y generar las gráficas; los resultados provienen de la ejecución del código.
Verificación	El código se ejecutó con semillas fijas y los resultados se contrastaron con una referencia independiente cuando fue posible (ver sección de resultados). [Completa antes de entregar: ejecuté el código en mi equipo, comprobé que los números coinciden y modifiqué / entendí cada parte del algoritmo.]

Referencias (APA 7.ª ed.)

Python Software Foundation. (s. f.). The Python standard library. https://docs.python.org/3/library/

Russell, S., y Norvig, P. (2020). Inteligencia artificial: Un enfoque moderno (4.ª ed.). Pearson.

Wirsansky, E. (2020). Hands-on genetic algorithms with Python. Packt.